Project - Tasks

Software Testing

The Software Testing project consists of six phases: five testing approaches plus one research component. Each student group will apply these techniques to their selected Python project.
Important: The main project code and tests will be pushed to a dedicated Git repository on the TUHH GitLab instance and gradually expanded with new unit tests as you progress through each phase. However, the main documentation — including all Markdown answer files and supplementary material — must be uploaded as specified under each task sheet in the corresponding submission folder on StudIP. A submission is only accepted via that folder, and it is only accessible before the project deadline.

StudIP submission folders.

- **Submissions (EX, R, ST)** – Individual submissions for exercise sessions, research tasks, and skill tests. One subfolder per type (`EX01`, `EX02`, ...).
- **Submissions (P)** – Group project submissions. One subfolder per task (`P01`–`P05`, `PRII`, `PRIII`), each containing one subfolder per group. Multiple submission versions per group are allowed.
  Example: `Submissions (P)` → `P01` → `Group06` → `P01_Group06_V01.zip`

The overall course grading is based on **100 points**: the **Project (84P)** spans six tasks at 14 P each (Random Testing, Input Space Partitioning, Graph Coverage, Logic Coverage, Syntax Coverage, Research in Software Testing), the **Final Presentation (4P)**, active participation in **Exercise Sessions (11P)** (relevant weeks are listed in the introductory session), and the **Industry Talk (1P)** in Week 8.

Sub-deadlines for individual contributions—in particular the PR (Research) sub-tasks—are managed via StudIP. Please check the **StudIP announcements** and the **expiry dates of the respective StudIP submission folders** for the binding sub-deadlines.

DEADLINE:	July 2, 2026 (AoE)
	AoE = Anywhere on Earth

Each major task submission requires both **implementation** (source code pushed to your group's GitLab repository, worth 7 points) and **documentation** (a Markdown answer file, worth 7 points). **The commit history in the group's repository will be used to track and evaluate each member's specific contributions.** Consistent and substantial contributions from all team members are essential for individual grades. Upload and naming instructions are provided in the **Submissions** box at the end of each task.
Before starting the project tasks, students must first form a group and select a Python project—both are due by April 17, 2026 (AoE) as described below.

Preparation - Group Registration & Project Selection

Group Registration. Please register for a project group in the Software Testing 2026 Stud.IP course. The groups can be found under *Participants* → *Groups*. Groups consist of **5 students**.
To find group members, you can:

- Connect with other students during lectures
- Use the exercise sessions to meet potential team members
- Post in the Stud.IP forum to find teammates

If you have difficulties finding a group, please reach out to the course staff, and we will assist you in finding a suitable team.

Project Selection. For this course, your group selects a Python project that meets the following requirements:

a) Written in the Python programming language

b) Contains at least 10,000 lines of code

c) Must have pytest tests (required for coverage analysis tools)

Suitable projects can be found on **Awesome Python**,[1] **GitHub Explore**,[2] or **PyPI**.[3]

Once you have selected a project, contact Daniel Rashedi[4] with your project proposal by April 17, 2026 (AoE). Your proposal should demonstrate how the project meets all required criteria. The project selection must be approved before you can proceed with the course tasks.

***Deadline**: April 17th, 2026 (AoE)*

Task 1 — Random Testing [14 P]

Similar to Exercise 01, the first project phase deals with random testing using `hypothesis`. This time, you will apply the library to your selected Python project to generate test cases, and reflect on the results.

[1] https://github.com/vinta/awesome-python
[2] https://github.com/explore
[3] https://pypi.org/search/
[4] daniel.rashedi@tuhh.de

- Each *individual* group member selects their own methods from the group's Python project and writes **4 tests** applying `hypothesis` random input generation strategies.
- Provide screenshots of your terminals that indicate your execution of the tests and the test cases generated by `hypothesis`.
- As a group, select a test from one of the generated files, and briefly describe the sequence of input values generated by `hypothesis`. Do the tests represent meaningful scenarios (6 - 9 sentences)?

Each group member makes **individual commits** to the group repository for their test cases.

AI Review Task Recall RQ1 *How Understandable is the test Suite provided by ChatGPT?* from the paper *ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation* covered in Research Task 02:

- As a group, decide on methods of those selected above and instruct an LLM of arbitrary choice to generate at least **4 tests** using `hypothesis` for random testing.
- Assess the aspects of correctness and understandability of your own test cases in comparison to the LLM's generated test cases according to the paper's methodology described in its RQ1.
- Document your selected LLM, your prompts, the generated test cases, and your assessment.

Although only 1 group member may submit the documentation, each answer should be annotated to indicate the group member who contributed it.

Submissions

- Upload `P01_Group[YY]_V[ZZ].zip` containing your `*.md` documentation and `*.py` test files to `Submissions (P)` → `P01` → `Group[YY]`.
- The uploaded template must include a filled-out table of each group member's individual contributions to the sub-tasks as a reference for grading.

Task 2 — Input Space Partitioning [14 P]

After completing Exercise 02, apply input space partitioning to your project. During this phase, you will create Python tests derived from an input domain model.

a) Each group member selects a method of the group's Python project which is suitable for the input domain model approach. Motivate your choices (1-2 sentences).

b) Identify the input domain of your method (2-3 sentences).

c) Similar to **Exercise 02**, gather the results of the following sub-tasks in a table as shown in Table 1:

a) Identify reasonable characteristics of the possible input data

b) Derive sets of equivalence classes from these characteristics (Note: At least one of your partitions must consist of a number of blocks ≥ 3. Also make sure that you have enough characteristics and corresponding blocks to derive $6 * n$ distinct tests from them later on where n is the number of group members.)

d) Can you identify any non-valid combinations of blocks from your characteristics? (1-2 sentences)

e) Derive (at least) one representative value for each equivalence class. Which strategy did you use for the value selection?

f) Combine the selected values / value ranges from the partitions of each input argument to complete test vectors. Provide a listing in your documentation.

g) Using your list of test vectors, create $6 * n$ Python tests where n is the number of group members. For the submission, merge all these tests into one `*.py` file and **annotate** the tests according to the ISP cases they cover.

Each group member makes **individual commits** to the group repository for their test cases.

AI Review Task Recall the viewpoint analysis method as conducted in the paper *ChatGPT and Human Synergy in Black-Box Testing: A Comparative Analysis* from Research Task 02:

- As a group, choose a method from those previously selected and instruct an LLM of your choice to derive multiple characteristics with corresponding value blocks, and to generate test cases for these blocks using `hypothesis`.
- Conduct viewpoint analysis of the LLM's generated test cases according to the paper's methodology described in its RQ1.
- Document your selected LLM, your prompts, the generated test cases, and your analysis.

Although only 1 group member may submit the documentation, each answer should be annotated to indicate the group member who contributed it.

Submissions

- Upload `P02_Group[YY]_V[ZZ].zip` containing your `*.md` documentation and `*.py` test files to `Submissions (P)` → `P02` → `Group[YY]`.
- The uploaded template must include a filled-out table of each group member's individual contributions to the sub-tasks as a reference for grading.

Task 3 — Graph Coverage [14 P]

You will now apply the concepts of graph coverage to your projects. For the extension of your test suite, you are allowed to choose between two different

Characteristic	Block/Eq-Class 1	Block/Eq-Class 2	Block/Eq-Class 3	...
q_1 = "..."	A1	A2	A3	...
q_2 = "..."	B1	B2	B3	...
q_3 = "..."	C1	C2	C3	...
...	...	...	...	...

Table 1: Template characteristics and partitions table.

tasks, depending on your current project focus. As done in **Exercise 03**, using `pytest`'s coverage extension is again the recommended tool for the coverage analysis, but you are not bound to it, especially if you want to work with specific coverage criteria that are not supported by this tool.

a) Measure the coverage of your given project test suite (which includes the existing test suite as well as the tests that you created in previous project phases) by 2 graph coverage criteria which you can freely choose. Describe each individual result in 2-3 sentences.

b) Extend the test suite with own tests, which have to fulfil **one** of the following criteria:

   a) Increase the coverage values of both coverage criteria that you applied in the previous subtask with at least 6 tests per group member **annotated** with the cases they cover with respect to coverage (compare and describe the effects on coverage for each individual test), OR

   b) Reveal a new bug in the software project (describe the bug, its context, and a potential fix in detail)

Each group member makes **individual commits** to the group repository for their test cases. (NOTE: Make sure that you pick coverage criteria that do not already reach a coverage of 100%, so there is some room for your improvements)

AI Review Task Recall the *SymPrompt* approach described in the paper *Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM* from Research Task 04:

- As a group pick the methods selected by 1 group member in the previous task to improve coverage values for, and manually apply the *SymPrompt* approach for prompt engineering on the selected parts of the project.
- Evaluate the LLM's generated test cases according to the paper's methodology described in its RQ1. List the resulting coverage value changes for each individual test case.
- Document your selected LLM, your prompts, the generated test cases, and your evaluation.

Although only 1 group member may submit the documentation, each answer should be annotated to indicate the group member who contributed it.

Submissions

- Upload `P03_Group[YY]_V[ZZ].zip` containing your `*.md` documentation and `*.py` test files to `Submissions (P)` → `P03` → `Group[YY]`.
- The uploaded template must include a filled-out table of each group member's individual contributions to the sub-tasks as a reference for grading.

Task 4 — Logic Coverage [14 P]

Similar to the graph coverage approach of the previous task, you will now apply the concepts of logic coverage to the project.

a) Pick n methods (n = number of group members) of your project which contains at least **2** predicates, with one of these predicates containing at least **3** clauses. If you cannot find a three-clause-predicate in your code, you can decide to take a nested structure of *if-else* and/or nested *while* expressions instead (at least **2** levels of nesting). In that case, explain how the application of your logic coverage criteria differs for nested structures as compared to "flat" predicates.

b) Using the *pytest* coverage extension from prior tasks, measure and note down the *branch* and *line coverage* values of the existing test suite regarding your selected method. Include a screenshot of the coverage report.

c) Create a set of tests that satisfy n of the following logic criteria (pick one concrete criterion from each of your n selected bullet points):

- *Predicate Coverage* (PC), *Clause Coverage* (CC), *Combinatorial Coverage* (CoC)
- *Active Clause Coverage* (ACC), *General Active Clause Coverage* (GACC), *Restricted Active Clause Coverage* (RACC), *Correlated Active Clause Coverage* (CACC)
- *Inactive Clause Coverage* (ICC), *General Inactive Clause Coverage* (GICC), *Restricted Inactive Clause Coverage* (RICC)
- *Implicant Coverage* (IC)
- *Multiple Unique True Point Coverage* (MUTP), *Unique True Point and Near False Point Pair Coverage* (CUTPNFP), *Multiple Near False Point Coverage* (MNFP)

d) **Annotate** each individual test the coverage criterion / criteria it contributes to (with comments).

e) Finally, recheck your method with the new, extended test suite regarding the *instruction*, *branch* and *line coverage* criteria, and explain *how* their values have / have not changed.

Each group member makes **individual commits** to the group repository for their test cases. (NOTE: Try to pick a method that does not already reach a coverage of 100% for branch and line, if possible.)

Abbr.	Mutation Operator Name	Mutation (a is a constant or expression)
ABS	Absolute Value Insertion	$a \rightarrow abs(a), -abs(a), failOnZero(a)$
UOI	Unary Operator Insertion	$a \rightarrow \{!, -, +, \sim\}a$
UOD	Unary Operator Deletion	$\{!, -, +, \sim\}a \rightarrow a$
AOR	Arithmetic Operator Replacement	$a + b \rightarrow a\{-, *, /, **, \%\}b, a, b$
ROR	Relational Operator Replacement	$a > b \rightarrow a\{>=, <, <=, ==, != \}b, true, false$
COR	Conditional Operator Replacement	$a\&\&b \rightarrow a\{\|\|, \&, \|, \hat{}\}b, true, false, a, b$
SOR	Shift Operator Replacement	$a << b \rightarrow a\{>>, >>>\}b, a$
LOR	Logical Operator Replacement	$a\&b \rightarrow a\{\|, \hat{}\}b, a, b$
ASR	Assignment Operator Replacement	$a+ = b \rightarrow a\{- =, * =, / =, \% =, \& =, \| =, \hat{}=, <<=, >>=, >>>=\}b$
SVR	Scalar Variable Replacement	$a = b * c \rightarrow a = b * b, a = b * a, a = a * c, a = c * c, b = b * c, c = b * c$
BSR	Bomb Statement Replacement	instruction $\rightarrow$ Bomb() (throws exception when reached)

Table 2: Exemplary mutation operators with their introduced changes for an expression / instruction (the operators are taken from *Introduction to Software Testing* by Ammann and Offutt)

Note: This phase has no AI Review sub-task. The Differential Prompting task (based on Research Paper 05) is dropped, as Paper 05 falls outside the adjusted course schedule.

Submissions

- Upload `P04_Group[YY]_V[ZZ].zip` containing your `*.md` documentation and `*.py` test files to `Submissions (P)` → `P04` → `Group[YY]`.
- The uploaded template must include a filled-out table of each group member's individual contributions to the sub-tasks as a reference for grading.

Task 5 — Syntax Coverage [14 P]

In this task, you will apply individually chosen criteria for syntax-based coverage to your software project. As before, gather all tests of the original project test suite, as well as the tests you created with the *Random Testing*, *Input Space Partitioning*, *Graph-based Coverage*, and *Logic-based Coverage* approaches. As part of this task you will create a model of a method or class of your project in Uppaal, and apply formal requirements to it. For this purpose, download the latest stable version of Uppaal from `http://uppaal.org/` and follow the instructions below.

a) As a group, decide on **one mutation tool** for Python code. The tools may be picked from the following list, but are not limited to them:
   a) *Mutmut* (`https://github.com/boxed/mutmut`)
   b) *Cosmic Ray* (`https://github.com/sixty-north/cosmic-ray`)
   c) *MutPy* (`https://github.com/mutpy/mutpy`)
   d) *Mutatest* (`https://github.com/EvanKepner/mutatest`)
   e) *PyMuTester* (`https://github.com/naver/PyMuTester`)

b) Depending on the capabilities of the selected mutation testing tool, determine the number of *killed mutants* for at least n **different mutation operators** where n is the number of group members.

c) For each of the applied mutation operators, pick a distinct method of your project that was affected by that operator, and highlight / describe the instructions that are changed. Explain how the test suite does or does not satisfy the criteria of *Reachability*, *Infection*, and *Propagation* for that part.

Next, building on your understanding of how mutations can alter program behavior, you will now create a formal model to verify program correctness against potential deviations.

a) Pick one suitable method (restricted to int values) or class from your project, and derive an automaton expressing its behaviour. For the creation of your automaton, use the *Uppaal* model checker environment. Include a *screenshot* of the model in your documentation.

b) Formulate at least 2 formal requirements that your model / method needs to satisfy, create corresponding *queries* in Uppaal, and execute them to assure that your base model satisfies them. Enable `Options` → `Diagnostic Trace` → `Some` to get a counterexample trace in case that a queried formula is not satisfied. This trace can be inspected in the `Simulator` tab.

c) Apply **one mutation operator** from Table 2 to **one expression** in your model, and re-evaluate your queries. Explain why the verification results changed or remained the same. If a counterexample is generated, note the test vector that kills the mutant.

d) Reroute one transition in your model (i.e., connect the edge to a different destination node), and re-evaluate your queries. Explain why the verification results of the model changed or remained the same. If a counterexample is created, note the test vector that kills the mutant.

Although only 1 group member may submit the documentation, each answer should be annotated to indicate the group member who contributed it.

Submissions

- Upload `P05_Group[YY]_V[ZZ].zip` containing your `*.md` documentation and `*.py` test files to `Submissions (P)` → `P05` → `Group[YY]`.
- The uploaded template must include a filled-out table of each group member's individual contributions to the sub-tasks as a reference for grading.

Task R - Research in Software Testing [14 P]

This task is split into three sub-parts (PR-I, PR-II, PR-III) with separate deadlines.

PR-I: Paper Selection.

- Read through the lists of accepted papers for the following conferences, and pick a paper which catches your interest:

  - IEEE International Conference on Software Testing, Verification and Validation 2025 (**ICST 2025**).[5]
  - ACM SIGSOFT International Symposium on Software Testing and Analysis 2025 (**ISSTA 2025**).[6]

- Check the **Wiki** entry on paper selection in the Software Testing course on StudIP (`Wiki` → `QuickLinks` → `Research Task - Paper Selection`). If your paper was not selected by another student yet, add your choice to the table (click `Edit`, and `Save` afterwards). Otherwise, pick another paper.

PR-II: Presentation.

- Hold a roughly 15-min presentation on your selected paper during the exercise session. The presentation will take place in two rounds depending on your regular time slot; the exact dates for each slot are announced via StudIP.
- Your presentation should cover: problem description, approach, and results.

PR-III: AI Review Task.

a) For a core section of your paper (e.g. approach section), let an AI tool provide a summary of 400–500 words.

b) Compare the AI-generated summary with the original text and evaluate its quality regarding correctness, completeness, and potential peculiarities (hallucinations, overgeneralisations, misleading statements).

c) Select one of the five research exercise papers covered during the course and one column of guiding questions from the corresponding exercise sheet.

d) Let an LLM answer the questions from the selected column. Compare the LLM's answers with your own answers (if available), or evaluate the correctness and overall quality of the LLM's answers.

Submissions

- **PR-I**: Register your paper in the StudIP Wiki.
- **PR-II**: Upload presentation slides (PDF only) as `PR_Group[YY]_V[ZZ].zip` to `Submissions (P)` → `PRII` → `Group[YY]`.
- **PR-III**: Upload your answers as `PR_Group[YY]_V[ZZ].zip` to `Submissions (P)` → `PRIII` → `Group[YY]`.
- *Deadlines for the individual sub-tasks (PR-I, PR-II, PR-III) are managed via StudIP. Please refer to the StudIP announcements and to the expiry dates of the respective StudIP submission folders for the binding sub-deadlines.*
- All templates must include a filled-out table of each group member's individual contributions as a reference for grading.

[5] Accepted papers: https://conf.researchr.org/track/icst-2025/icst-2025-papers. Papers behind a paywall can often be found via Semantic Scholar (https://www.semanticscholar.org/) or arXiv (https://arxiv.org/).

[6] Proceedings: https://dl.acm.org/doi/proceedings/10.1145/3713081

Presentation - Final Presentation **[4 P]**

As part of the course assessment, each group will give a final presentation covering challenges and insights in applying the five testing techniques to their selected projects. Presentations will be held during the last exercise sessions.

- Prepare slides of **at most 10 pages**.
- Cover challenges, insights, and lessons learned for each of the 5 testing approaches (Random Testing, ISP, Graph-Based, Logic-Based, Syntax-Based) as applied to your project.
- We will **randomly select** which group member presents each section. Every group member must be prepared to present **any part** of the presentation.

Submissions

Upload your presentation slides (PDF only) to `Submissions (P)`. Detailed submission instructions will be announced in due time.